深度学习编译器实战

--以Triton为例

编译实战培训班

讲师团介绍

先进编译实验室
ADVANCED COMPILING LABORATORY

李帅龙

- 先进编译实验室高性能计算部部长
- FlagGems&FlagTree开源项目贡献者
- 主研算子开发、性能优化

李嘉楠

- 先进编译实验室副主任
- 主研人工智能编译器、编译优化
- 多项开源AI项目负责人
- 2025智源大会Tech Tutorials技术分享者

王磊

- 先进编译实验室研究员
- B站《AI实操优化-Triton编译器》主讲人
- 手机深度学习编译优化

公众号：先进编译实验室

- **Triton是什么**

  Triton的全面开源特性为AI芯片厂商的适配工作，其特点使得开发者可以根据自研芯片的架构特征和独有的硬件设计来灵活调整编译器的行为

- **适合人群**
  - 计算机应届毕业生
  - 想转行/跨行人员
  - IT相关工作者

- **课程优势**
  - 理论+实践
  - 一对一指导
  - 聚焦前沿的技术成果

课程目录

- 课程环境配置&Triton与PyTorch
- Triton Kernel性能分析工具
- Triton语言特征详解及关键参数优化
- Triton编译器及Pass管理器
- Triton关键属性Layout介绍
- 编译优化实践(一)：代数变换pass添加
- 编译优化实践(二)：冗余去除pass添加

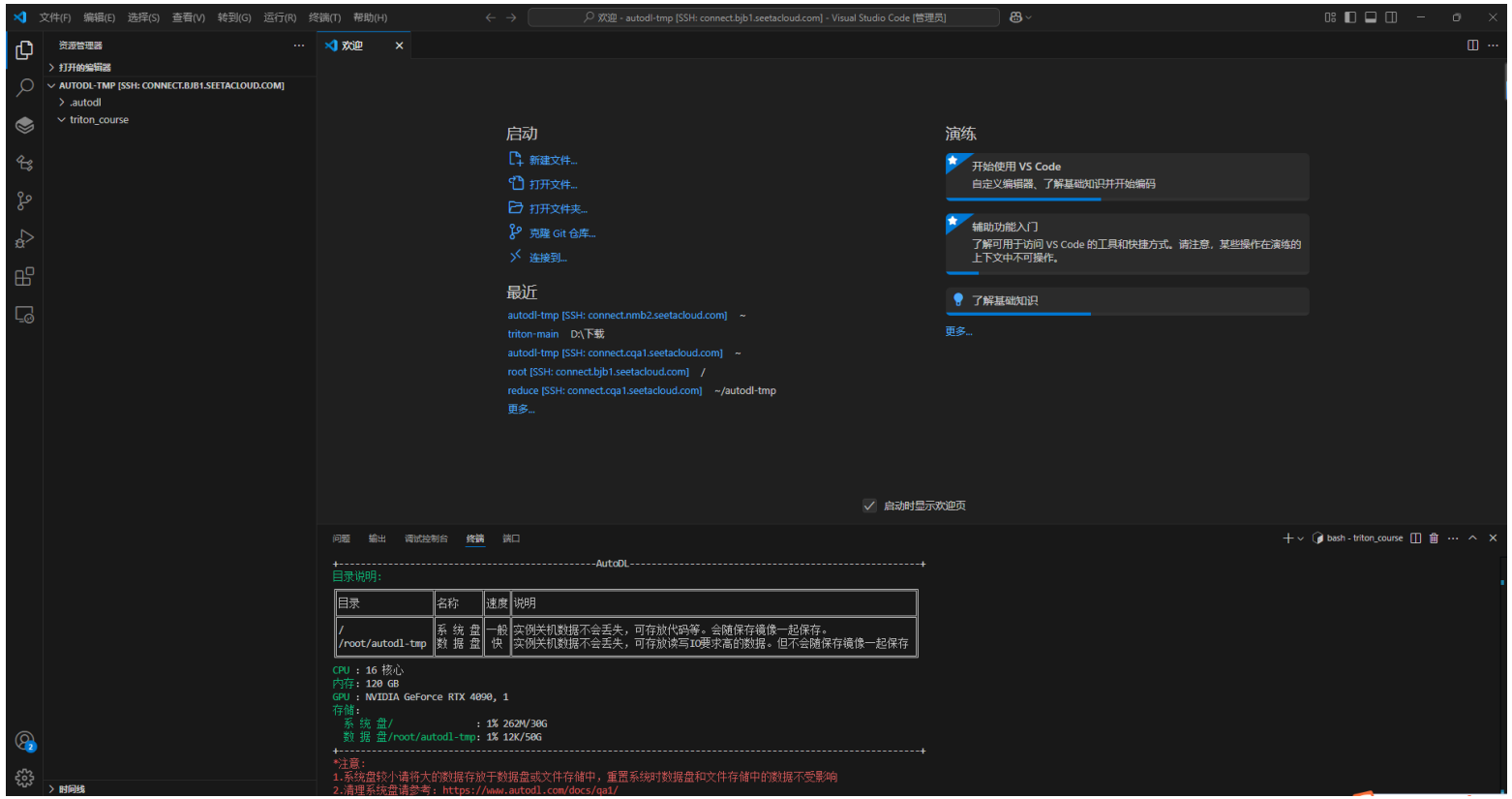
第一讲

课程环境配置/Triton与PyTorch

课程环境配置

1、登录指令 ssh -p 54419 root@connect.bjb1.seetacloud.com

2、本地使用VSCode或Xshell远程连接后



课程环境配置

已安装python包查看

```
pip list
```

学术资源加速

```
source /etc/network_turbo
unset http_proxy && unset https_proxy
```

Notebook中使用

```
import subprocess
import os
result = subprocess.run('bash -
c "source /etc/network_turbo && env | grep pr
oxy"', shell=True, capture_output=True, text=
True)
output = result.stdout
for line in output.splitlines():
    if '=' in line:
        var, value = line.split('=', 1)
        os.environ[var] = value
```

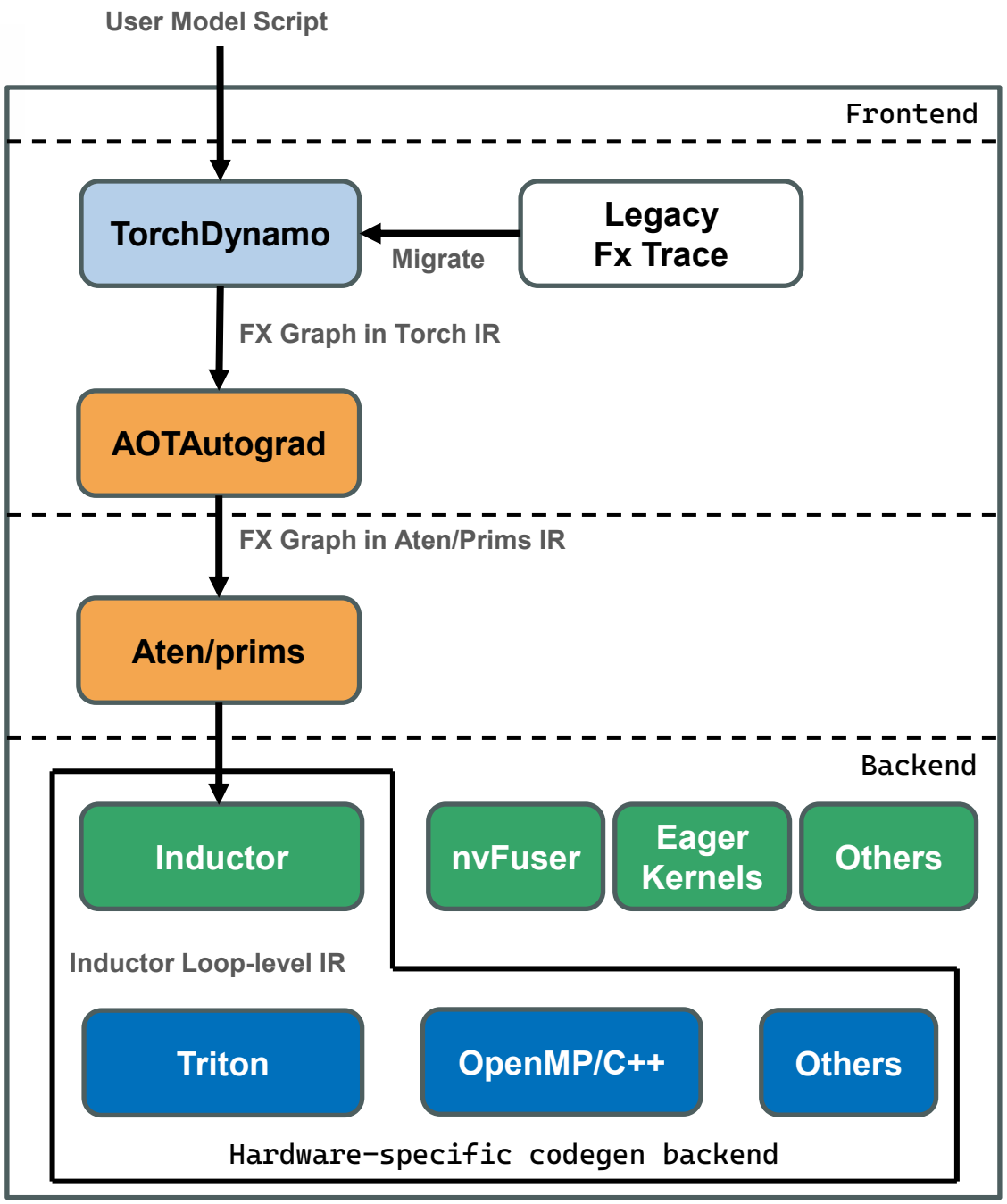
conda虚拟环境配置

```
# 构建一个虚拟环境，名为env_name
conda create -n env_name python=3.7
# 删除指定虚拟环境
conda remove -n env_name --all
# 更新bashrc中的环境变量
conda init bash && source /root/.bashrc
# 切换到创建的虚拟环境env_name
conda activate env_name
# 关闭当前虚拟环境
conda deactivate
# 查看已有的虚拟环境
conda env list
# 查看指定环境下已安装的包
conda list -n env_name
# 安装指定环境下某个包
conda install -n env_name [package]
# 删除指定环境下某个包
conda remove -n env_name [package]
# 更新指定环境下某个包
conda update -n env_name [package]
# 关闭打开终端自动进入虚拟环境
conda config --set auto_activate_base false
```

PyTorch2.0介绍

PyTorch2.0编译模式

Pytorch2.0在保留原有优势的同时（提供了相同的eager mode和用户体验），通过torch.compile增加了编译模式，支持在训练和推理过程中对模型进行加速，从而提供更优的性能。



PyTorch2.0关键技术组件

PrimTorch

通过定义更小更稳定的算子集Aten ops和Prim ops，让PyTorch程序可以持续降级至这些算子集，来解决因算子数量过多编写PyTorch后端复杂的问题。

TorchInductor

TorchInductor可以进行算子融合并自动生成低级的GPU上的Triton代码以及CPU上的C++/OpenMP，相较Eager模式内核启动较少，内存读取/写入更少，从而提升性能

01 02 03 04

高效的图捕获工具，将所有复杂算子减少到PrimTorch中的约250个原始算子，同时移除未使用的算子，确定需要存储或写入内存的中间算子以及可以融合的算子，减小模型开销。

TorchDynamo

AOTAutograd使得开发者得以提前捕获反向传播，基于完整的前向和反向图视角，根据依赖关系进行算子调度，对算子和层进行融合。

AOTAutograd

TorchInductor编译栈

AotAutograd

Decomposes into smaller operator set

Capture forwards + backwards

Some inductor specific decomps included in this step

Inductor Graph Lowerings

Remove views, broadcasting, and simplify indexing

Rematerialize vs reuse decisions

Layout tuning and optimization

Loop order

Inductor Scheduling

Horizontal / vertical fusion decisions

Reduction fusions

Tiling

Memory planning and buffer reuse

In-place memory buffers

Autotuning / kernel selection

Wrapper Codegen

Outer code that calls kernels and allocates memory

(Replaces interpreter)

Backend Codegen

Triton

C++

PyTorch2.0流程实践

使用pip安装torch相关套件

```
pip install torch torchvision torchaudio
```

PyTorch 2.0 编译API

```
def compile(model: Optional[Callable] = None, *,
            fullgraph: builtins.bool = False,
            dynamic: Optional[builtins.bool] = None,
            backend: Union[str, Callable] = "inductor",
            mode : Union[str, None] = None,
            options: Optional[Dict[str, Union[str, builtins.int, builtins.bool]]] = None,
            disable: builtins.bool = False) -> Callable:
```

torch.compile 选项

model：指定需要编译的函数或模型

fullgraph：指定是否可将模型拆分为子图

dynamic：指定尽可能生成生成动态Kernel以适应shape变化避免重新编译

backend：用于指定后端，默认为Inductor

mode：包括default、reduce-overhead、max-autotune、max-autotune-no-cudagraphs等模式

- reduce-overhead模式是通过使用CUDA graph来减小开销，适用于小batch
- max-autotune模式是对matmul、conv2d等算子进行配置优化，并与库函数的性能权衡选择更优
- max-autotune-no-cudagraphs模式是对matmul、conv2d等算子权衡Triton性能，不使用CUDA graph

option：

- epilogue_fusion：max_autotune模式下融合pointwise算子至模版
- max_autotune：选择最优的矩阵配置
- fallback_random：适用于精度问题调试
- shape_padding：填充矩阵shape以对齐load
- triton.cudagraph：使用CUDA graph来减小python开销
- trace.enabled：调试开关
- trace.graph_diagram：查看融合后的图

disable：将torch.compile()转换为空操作进行测试

torch.compile 使用方法

```
def foo(x, y):
    a = torch.sin(x)
    b = torch.cos(y)
    return a + b
opt_foo1 = torch.compile(foo)
print(opt_foo1(torch.randn(10, 10), torch.randn(10, 10)))
```

将优化的函数/模型传递给torch.compile

```
t1 = torch.randn(10, 10)
t2 = torch.randn(10, 10)

@torch.compile
def opt_foo2(x, y):
    a = torch.sin(x)
    b = torch.cos(y)
    return a + b
print(opt_foo2(t1, t2))
```

使用@torch.compile装饰要优化的函数/模型

torch.compile 使用方法

执行文件

```
TORCH_COMPILE_DEBUG=1 python xx.py
```

转储文件路径

```
W0227 16:41:45.213000 30267 site-packages/torch/_inductor/debug.py:435] [0/0] model__0_inference_0 debug trace: /r
oot/autodl-tmp/BERT_pytorch/torch_compile_debug/run_2025_02_27_16_41_35_961775-pid_30267/torchinductor/model__0_in
ference_0.0
```

缓存清理

```
rm -rf /tmp/torchinductor_xx
```

torch.compile 使用方法

执行文件

```
TORCH_COMPILE_DEBUG=1 python xx.py
```

转储文件路径

```
W0227 16:41:45.213000 30267 site-packages/torch/_inductor/debug.py:435] [0/0] model__0_inference_0 debug trace: /r
oot/autodl-tmp/BERT_pytorch/torch_compile_debug/run_2025_02_27_16_41_35_961775-pid_30267/torchinductor/model__0_in
ference_0.0
```

fx_graph_runable.py：分解后，模式匹配前的FX图

fx_graph_transformed.py：模式匹配后的FX图

ir_pre_fusion.txt：融合前的Inductor IR

ir_post_fusuion.txt：融合后的Inductor IR

output_code.py：生成带有C++或Triton Kernel的Python代码

编译生成Triton Kernel

Input Code	FX Graph IR	ATen FX Graph IR	Define-by-run IR	Scheduling/Fusion	Output Triton	Output Wrapper

```
import
def toy_example(x):
    y = x.sin()
    z = y.cos()
    return z
x = torch.randn(1000, device="cuda", requires_grad = True)
compiled_f = torch.compile(toy_example, backend='inductor')
output = compiled_f(x)
```

查看编译流程中的一些中间过程/中间表示

```
TORCH_LOGS="dynamo,aot,+inductor,bytecode,graph,graph_code,aot_graphs,aot_joint_graph,sch
edule,fusion,output_code" python test_compiler.py
```

编译生成Triton Kernel

Input Code	**FX Graph IR**	ATen FX Graph IR	Define-by-run IR	Scheduling/Fusion	Output Triton	Output Wrapper

```
def forward(self, L_x_: "f32[1000]"):
    l_x_ = L_x_
    # File: /BERT_pytorch/test_compiler.py:4 in toy_example, code: y = x.sin()
    y: "f32[1000]" = l_x_.sin();  l_x_ = None

    # File: /BERT_pytorch/test_compiler.py:5 in toy_example, code: z = y.cos()
    z: "f32[1000]" = y.cos();  y = None
    return (z,)
```

编译生成Triton Kernel

Input Code	FX Graph IR	**ATen FX Graph IR**	Define-by-run IR	Scheduling/Fusion	Output Triton	Output Wrapper

```python
===== Forward graph 0 =====
def forward(self, primals_1: "f32[1000][1]cuda:0"):
    # File: /BERT_pytorch/test_compiler.py:4 in toy_example, code: y = x.sin()
    sin: "f32[1000][1]cuda:0" = torch.ops.aten.sin.default(primals_1)

    # File: /BERT_pytorch/test_compiler.py:5 in toy_example, code: z = y.cos()
    cos: "f32[1000][1]cuda:0" = torch.ops.aten.cos.default(sin);  sin = None

    return (cos, primals_1)
```

python

```python
 ===== Backward graph 0 =====
def forward(self, primals_1: "f32[1000][1]cuda:0", tangents_1: "f32[1000][1]cuda:0"):

    # File: /BERT_pytorch/test_compiler.py:4 in toy_example, code: y = x.sin()
    sin: "f32[1000][1]cuda:0" = torch.ops.aten.sin.default(primals_1)

    # File: /BERT_pytorch/test_compiler.py:5 in toy_example, code: z = y.cos()
    sin_1: "f32[1000][1]cuda:0" = torch.ops.aten.sin.default(sin);  sin = None
    neg: "f32[1000][1]cuda:0" = torch.ops.aten.neg.default(sin_1);  sin_1 = None
    mul: "f32[1000][1]cuda:0" = torch.ops.aten.mul.Tensor(tangents_1, neg);  tangents_1 =
neg = None

    # File: /BERT_pytorch/test_compiler.py:4 in toy_example, code: y = x.sin()
    cos_1: "f32[1000][1]cuda:0" = torch.ops.aten.cos.default(primals_1);  primals_1 =
None
    mul_1: "f32[1000][1]cuda:0" = torch.ops.aten.mul.Tensor(mul, cos_1);  mul = cos_1 =
None

    return (mul_1,)
```

编译生成Triton Kernel

Input Code	FX Graph IR	ATen FX Graph IR	**Define-by-run IR**	Scheduling/Fusion	Output Triton	Output Wrapper

```
def inner_fn_buf0(index):
    i0, i1 = index
    tmp0 = ops.load(arg0_1, i1 + i0 * s1)
    tmp1 = ops.sin(tmp0)
    return tmp1
def inner_fn_buf0(index):
    i0, i1 = index
    tmp0 = ops.load(buf0, i1 + i0 * s1)
    tmp1 = ops.cos(tmp0)
    return tmp1
```

```
buf0_ir = TensorBox(StorageBox(ComputedBuffer(
    name='buf0',
    layout=FixedLayout('cuda:0', torch.float32,
                       size=[1000], stride=[1]),
    data=Pointwise(device=device(type='cuda', index=0),
                   dtype=torch.float32,
                   inner_fn=inner_fn_buf0,
                   ranges=[1000]))))
buf1_ir = TensorBox(StorageBox(ComputedBuffer(
    name='buf1',
    layout=FixedLayout('cuda:0', torch.float32,
                       size=[1000], stride=[1]),
    data=Pointwise(device=device(type='cuda', index=0),
                   dtype=torch.float32,
                   inner_fn=inner_fn_buf1,
                   ranges=[1000]))))
```

编译生成Triton Kernel

Input Code	FX Graph IR	ATen FX Graph IR	Define-by-run IR	**Scheduling/Fusion**	Output Triton	Output Wrapper

torch/_inductor/scheduler.py

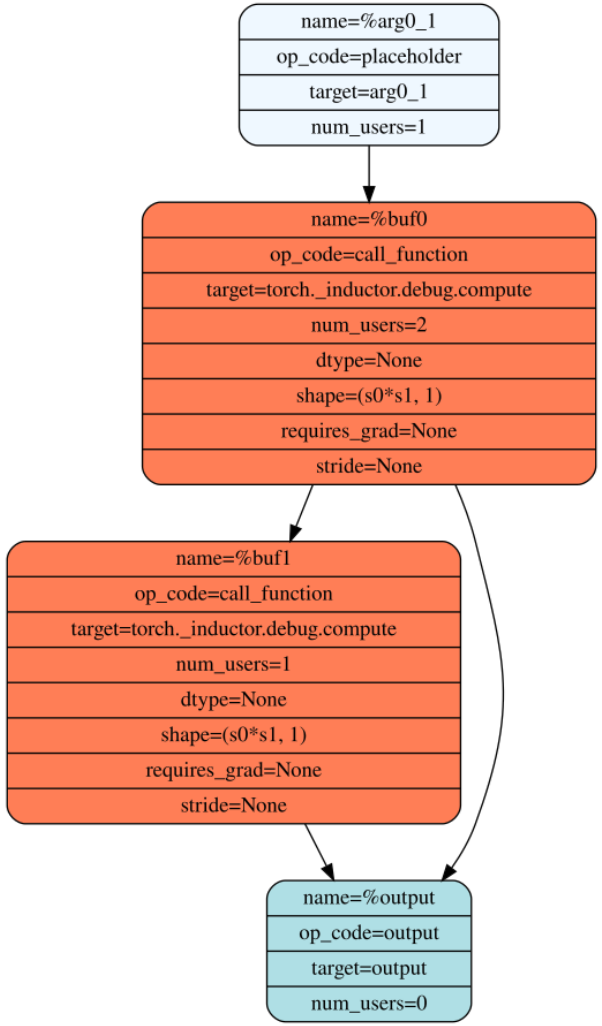
Scheduler.can_fuse(buf0, buf1)

True

Scheduler.score_fusion(buf0, buf1)

(True, True, 33554432, -1)

- True/True is category of fusion (pointwise+pointwise)
- 33554432 is estimated memory bandwidth saved by fusion: 8192*1024*4
- -1 is distance in input graph



编译生成Triton Kernel

Input Code	FX Graph IR	ATen FX Graph IR	Define-by-run IR	Scheduling/Fusion	**Output Triton**	Output Wrapper

```python
@triton.jit
def triton_poi_fused_cos_sin_0(in_ptr0, out_ptr0, xnumel, XBLOCK : tl.constexpr):
    xnumel = 1000
    xoffset = tl.program_id(0) * XBLOCK
    xindex = xoffset + tl.arange(0, XBLOCK)[:]
    xmask = xindex < xnumel
    x0 = xindex
    tmp0 = tl.load(in_ptr0 + (x0), xmask)
    tmp1 = tl_math.sin(tmp0)
    tmp2 = tl_math.cos(tmp1)
    tl.store(out_ptr0 + (x0), tmp2, xmask)
```

编译生成Triton Kernel

Input Code	FX Graph IR	ATen FX Graph IR	Define-by-run IR	Scheduling/Fusion	Output Triton	**Output Wrapper**

```python
def call(args):
    primals_1, = args
    args.clear()
    assert_size_stride(primals_1, (1000, ), (1, ))
    with torch.cuda._DeviceGuard(0):
        torch.cuda.set_device(0)
        buf0 = empty_strided_cuda((1000, ), (1, ), torch.float32)
        # Topologically Sorted Source Nodes: [y, z], Original ATen: [aten.sin, aten.cos]

        stream0 = get_raw_stream(0)
        triton_poi_fused_cos_sin_0.run(primals_1, buf0, 1000, grid=grid(1000),
stream=stream0)
    return (buf0, primals_1, )

def benchmark_compiled_module(times=10, repeat=10):
    from torch._dynamo.testing import rand_strided
    from torch._inductor.utils import print_performance
    primals_1 = rand_strided((1000, ), (1, ), device='cuda:0', dtype=torch.float32)
    fn = lambda: call([primals_1])
    return print_performance(fn, times=times, repeat=repeat)

if __name__ == "__main__":
    from torch._inductor.wrapper_benchmark import compiled_module_main
    compiled_module_main('None', benchmark_compiled_module)
```

Inductor生成Triton Kernel的性能表现

- fp16 Huggingface的45个模型平均性能优于eager模式

	Inference	Training
All TorchInductor optimizations	1.91×	1.45×
Without loop/layout reordering	1.91× (-0.00)	1.28× (-0.17)
Without matmul templates	1.85× (-0.06)	1.41× (-0.04)
Without parameter freezing	1.85× (-0.06)	1.45× (-0.00)
Without pattern matching	1.83× (-0.08)	1.45× (-0.00)
Without cudagraphs	1.81× (-0.10)	1.37× (-0.08)
Without fusion	1.68× (-0.23)	1.27× (-0.18)
Without inlining	1.58× (-0.33)	1.31× (-0.14)
Without fusion and inlining	0.80× (-1.11)	0.59× (-0.86)

Torchbench模型测试

Torchbench是由 PyTorch 团队创建的基准测试框架，旨在帮助开发者评估和比较不同模型在 PyTorch 上的性能，它提供了一些常见的TorchBenchmark、HuggingFace、TIMM等深度学习模型库和基准测试。

安装Torchbench

使用pip安装相关依赖

```
pip install pandas psutil pyyaml scipy tqdm tabulate
```

安装benchmark，可以通过选项安装指定模型

```
git clone https://github.com/pytorch/benchmark
cd benchmark
python install.py models=" BERT_pytorch"
pip install -e .
```

Torchbench模型测试选项

进行模型测试

```
TORCHINDUCTOR_UNIQUE_KERNEL_NAMES=1 TORCHINDUCTOR_BENCHMARK_KERNEL=1 python
 -u benchmark/userbenchmark/dynamo/dynamobench/torchbench.py --
backend inductor --float16 --performance --only BERT_pytorch --training
```

TORCHINDUCTOR_UNIQUE_KERNEL_NAMES=1：启动唯一的Kernel名称，方便调试分析

TORCHINDUCTOR_BENCHMARK_KERNEL=1：启动内核基准测试，在运行时记录Kernel性能数据

-u：将日志或输出打印至控制台

benchmarks/dynamo/torchbench.py：执行TorchBenchmark基准测试的脚本，其他两类模型库的脚本对应为benchmarks/dynamo/huggingface.py、benchmarks/dynamo/timm_models.py

-backend inductor：指定Inductor作为后端执行引擎

-float16：指定计算的精度为float16，其他同类选项还包括bfloat16、float32、amp（自动混合精度）

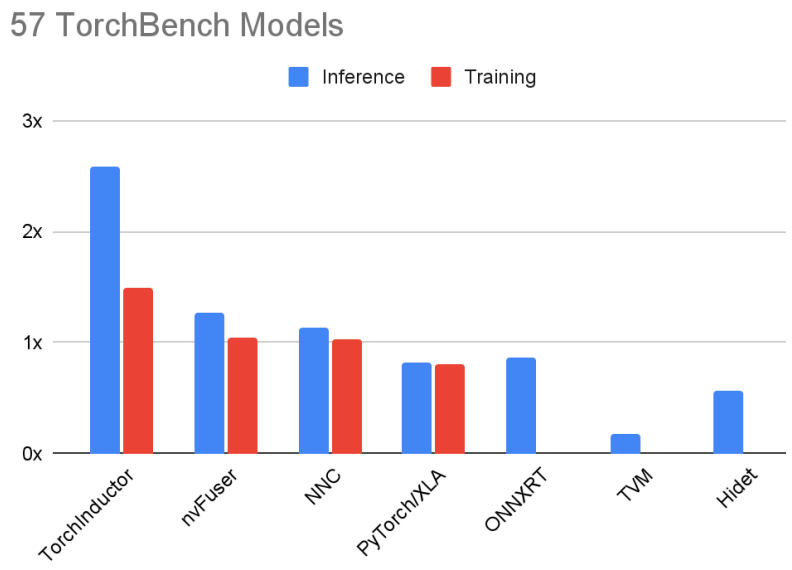
-performance：启动性能基准测试模式，其他同类选项还包括accuracy（正确性测试）

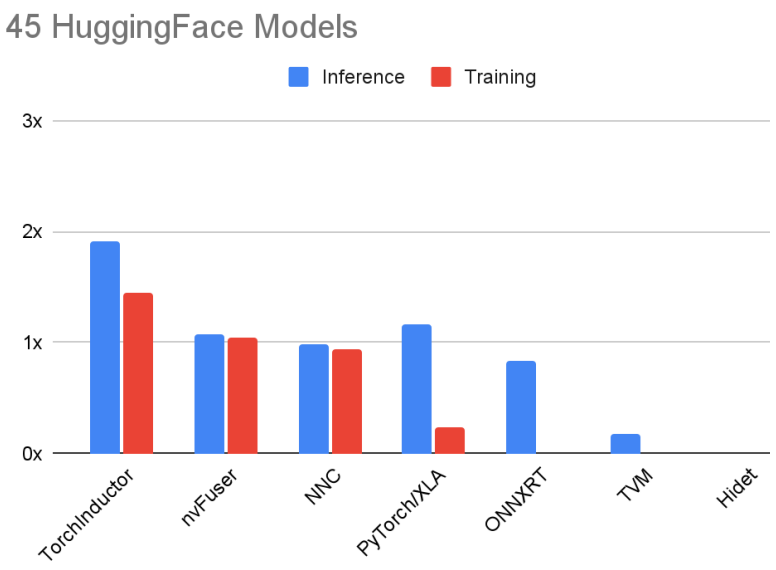
-only BERT_pytorch：仅运行BERT_pytorch模型基准测试，如不指定则会运行脚本中所有可用模型

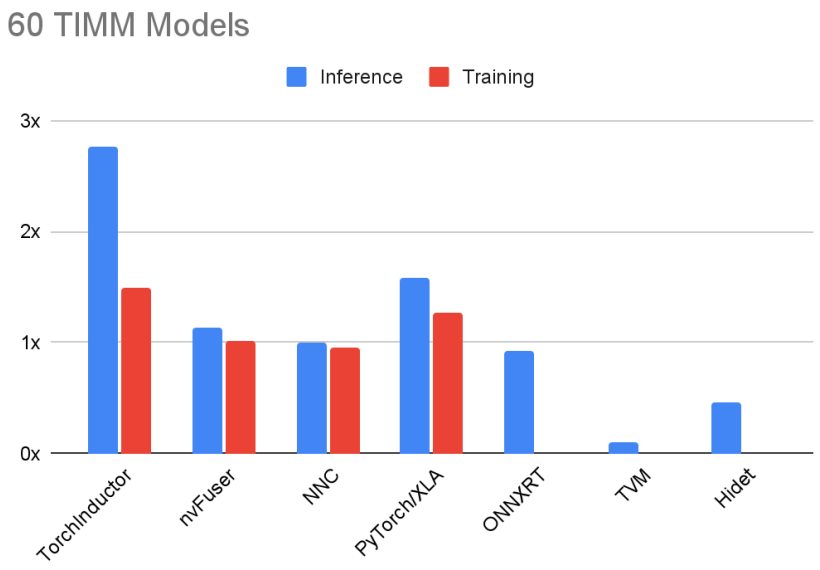
-training：启动训练模式，其他同类选项还包括inference（推理模式）

Inductor生成Triton Kernel的性能优势

Inductor相较其他编译器也具有一定性能优势







Inductor生成Triton Kernel的其他优势

Softmax

```python
@torch.compile
def softmax(x):
    x_max = x.max(dim=1)[0]
    z = x - x_max[:, None]
    numerator = torch.exp(x)
    denominator = numerator.sum(dim=1)
    return numerator / denominator[:, None]
```

```python
@reduction(
    size_hints=[256, 16384],
    reduction_hint=ReductionHint.INNER,
    filename=__file__,
    meta={'signature': {0: '*fp16', 1: '*fp16', 2: 'i32', 3: 'i32'}, 'device': 0, 'device
)
@triton.jit
def triton_(in_ptr0, out_ptr1, xnumel, rnumel, XBLOCK : tl.constexpr, RBLOCK : tl.constex
    rnumel = 16384
    xoffset = tl.program_id(0) * XBLOCK
    xindex = xoffset + tl.arange(0, XBLOCK)[:, None]
    xmask = xindex < xnumel
    rbase = tl.arange(0, RBLOCK)[None, :]
    x0 = xindex
    _tmp3 = tl.full([XBLOCK, RBLOCK], 0, tl.float32)
    for roffset in range(0, rnumel, RBLOCK):
        rindex = roffset + rbase
        rmask = rindex < rnumel
        r1 = rindex
        tmp0 = tl.load(in_ptr0 + (r1 + (16384*x0)), rmask & xmask, eviction_policy='evict
        tmp1 = tl.exp(tmp0)
        tmp2 = tl.broadcast_to(tmp1, [XBLOCK, RBLOCK])
        tmp4 = _tmp3 + tmp2
        _tmp3 = tl.where(rmask & xmask, tmp4, _tmp3)
    tmp3 = tl.sum(_tmp3, 1)[:, None]
    for roffset in range(0, rnumel, RBLOCK):
        rindex = roffset + rbase
        rmask = rindex < rnumel
        r1 = rindex
        tmp5 = tl.load(in_ptr0 + (r1 + (16384*x0)), rmask & xmask, eviction_policy='evict
        tmp6 = tl.exp(tmp5)
        tmp7 = tmp6 / tmp3
        tl.store(out_ptr1 + (r1 + (16384*x0)), tmp7, rmask & xmask)
```

- **支持动态shape**
- **感知内存和寄存器压力，根据归约维度选择生persistent reduction 或非连续归约reduction**

Inductor生成Triton Kernel的其他优势

Templates

对于卷积、矩阵乘等计算，Inductor会使用预先手工写好的模板方式生成Triton，对于其他一些操作也会自动生成

```python
@torch.compile(mode="max-autotune")
def f(x, y, bias):
    out = torch.mm(x, y)
    return out, out.cos(), out + bias
```

```python
...
# rematerialize rm and rn to save registers
rm = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
rn = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
idx_m = rm[:, None]
idx_n = rn[None, :]
mask = (idx_m < M) & (idx_n < N)

# inductor generates a suffix
xindex = idx_n + (4096*idx_m)
tl.store(out_ptr0 + (tl.broadcast_to(xindex, mask.shape)), acc, mask)
tmp1 = tl.load(in_ptr2 + (tl.broadcast_to(idx_n, mask.shape)), mask, eviction_policy=
tmp0 = tl.cos(acc)
tmp2 = acc + tmp1
tl.store(out_ptr1 + (tl.broadcast_to(xindex, mask.shape)), tmp0, mask)
tl.store(out_ptr2 + (tl.broadcast_to(idx_n + (4096*idx_m), mask.shape)), tmp2, mask)
```

Inductor生成Triton Kernel的其他优势

For-each Kernels

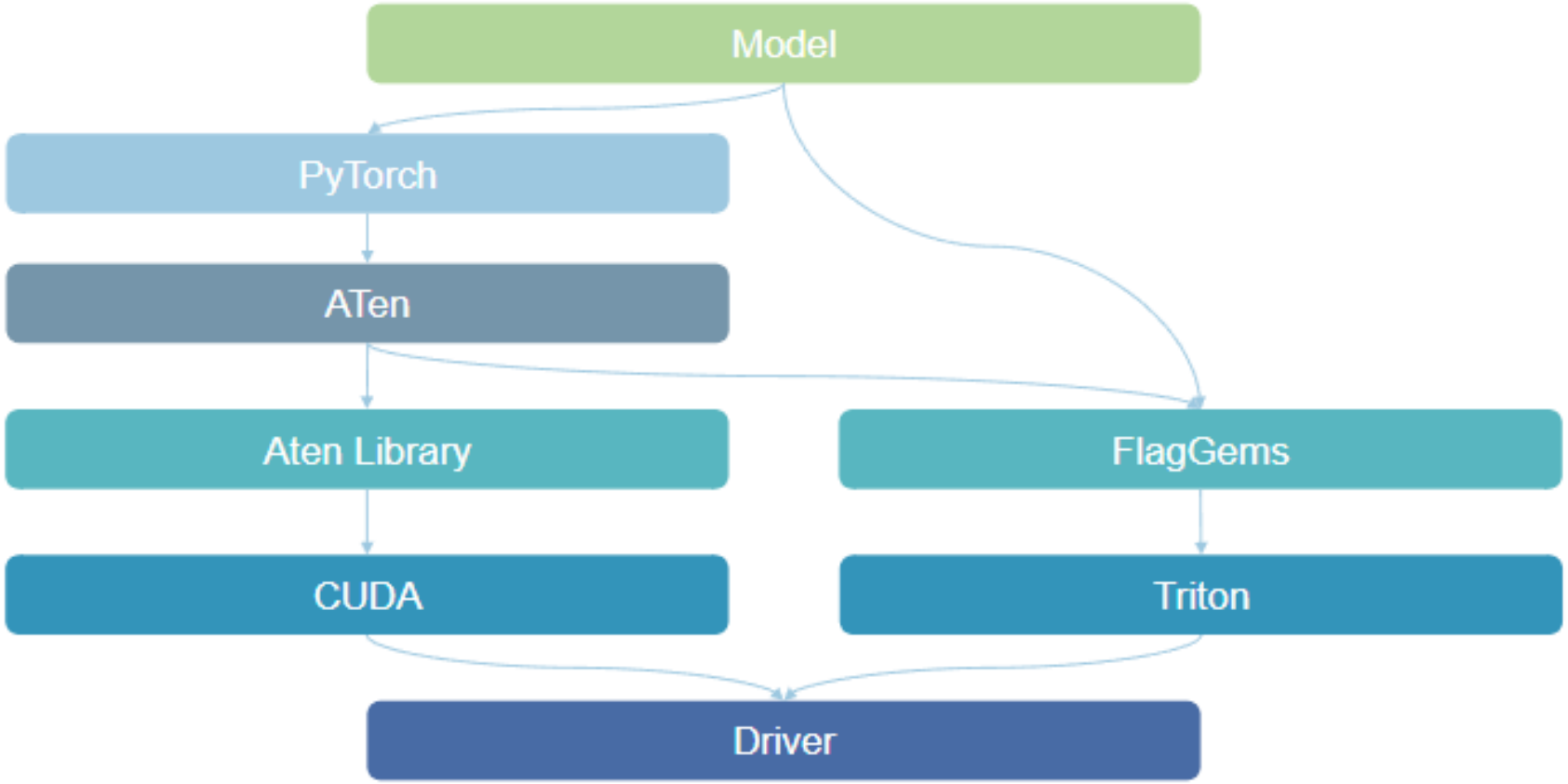
多个张量执行元素级操作foreach，Inductor利用Triton BLOCK级别控制流的抽象特性自动生成Triton Kernel，避免因每个 kernel 只启动少量的线程块而导致GPU占用率较低

```python
@torch.compile
def f(params):
    param1 = torch._foreach_add(params, 1)
    param2 = torch._foreach_mul(param1, params)
    return param2
f([torch.randn(i) for i in range(100, 150)])
```

```python
def triton_foreach(...):
    if xpid >= 0 and xpid < 1:
        xpid_offset = xpid - 0
        xnumel = 100
        xoffset = xpid_offset * XBLOCK
        xindex = xoffset + tl.arange(0, XBLOCK)[:]
        xmask = xindex < xnumel
        x0 = xindex
        tmp0 = tl.load(in_ptr0 + (x0), xmask)
        tmp1 = 1.0
        tmp2 = tmp0 + tmp1
        tmp3 = tmp2 * tmp0
        tl.store(out_ptr0 + (x0), tmp3, xmask)
    elif xpid >= 1 and xpid < 2:
        xpid_offset = xpid - 1
        xnumel = 101
        xoffset = xpid_offset * XBLOCK
        xindex = xoffset + tl.arange(0, XBLOCK)[:]
        xmask = xindex < xnumel
        x1 = xindex
        tmp4 = tl.load(in_ptr1 + (x1), xmask)
        tmp5 = 1.0
        tmp6 = tmp4 + tmp5
        tmp7 = tmp6 * tmp4
        tl.store(out_ptr1 + (x1), tmp7, xmask)
    elif xpid >= 2 and xpid < 3:
    ...
```

Triton算子库 FlagGems

使用OpenAI推出的Triton编程语言实现的**高性能通用算子库**，旨在为大语言模型提供一系列可应用于PyTorch框架的算子，加速模型的推理与训练。



https://github.com/FlagOpen/FlagGems

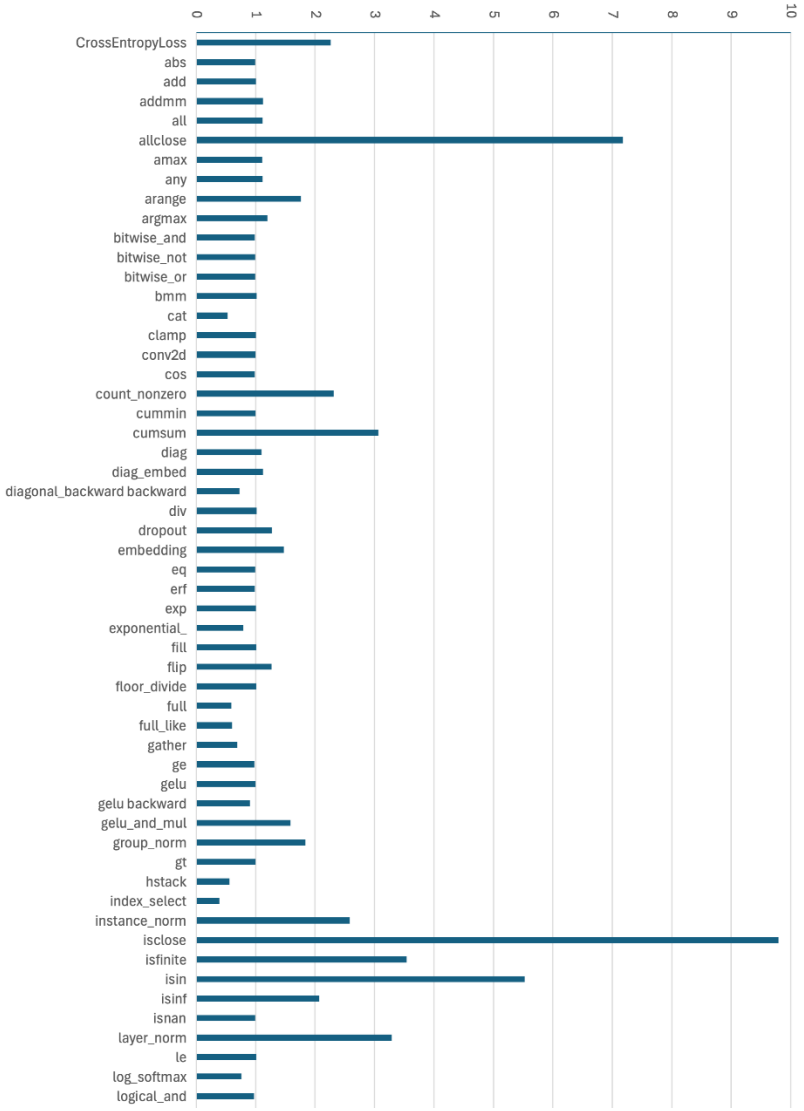
FlagGems的使用

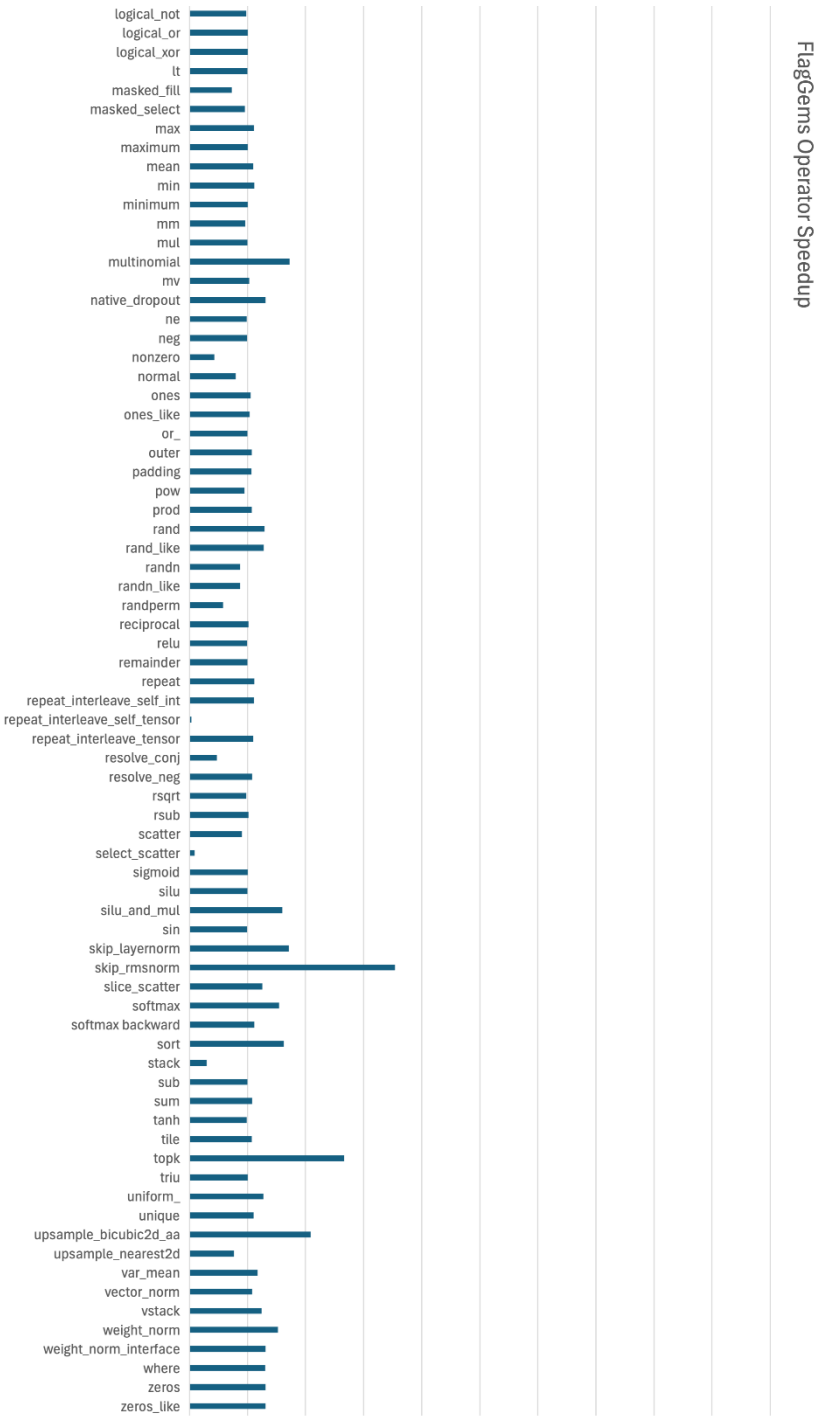
```python
import torch
import flag_gems
#方式一：在进程中永久启用
flag_gems.enable()

M, N, K = 1024, 1024, 1024
A = torch.randn((M, K), dtype = torch.float16, device = "cuda")
B = torch.randn((M, K), dtype = torch.float16, device = "cuda")
C = torch.mm(A, B)
```

```python
import torch
import flag_gems

M, N, K = 1024, 1024, 1024
A = torch.randn((M, K), dtype = torch.float16, device = "cuda")
B = torch.randn((M, K), dtype = torch.float16, device = "cuda")
#方式二：在进程局部临时调用
with flag_gems.use_gems():
    C = torch.mm(A, B)
```

```python
import torch
import flag_gems

M, N, K = 1024, 1024, 1024
A = torch.randn((M, K), dtype = torch.float16, device = "cuda")
B = torch.randn((M, K), dtype = torch.float16, device = "cuda")
#方式三：直接调用
C = flag_gems.mm(A, B)
```

FlagGems的性能表现





课程总结

1、掌握环境配置使用方法

掌握远程服务器连接，以及conda虚拟环境配置方法

2、掌握torch.compile编译模式使用

了解torch.compile的使用方法，对简单的程序/函数进行编译执行

3、了解torch.compile编译的过程

通过查看简单程序/函数编译过程中的中间过程/表示，并进行对比查看，了解torch.compile编译的过程

4、掌握TorchBench模型基准测试框架的使用

安装Torchbench基准测试框架，使用Torchbench运行模型生成Triton Kernel

5、（拓展）了解整理一下目前哪些场景下使用了Triton算子

先进编译实验室 Advanced Compilation Laboratory